The Luminary Heritage Resort

Property Management System (PMS) — Full Stack Architectural Specification

Document Type: System Architecture & Engineering Documentation | **Stack:** MERN (MongoDB, Express, React, Node) | **Version:** 2.4.0

Project Overview & Executive Summary

The Luminary Heritage Resort PMS is an enterprise-grade hospitality operating platform designed to orchestrate guest reservations, room inventory lifecycle synchronization, real-time stay-folio billing, and multi-cuisine in-room dining orders. Built on a decoupled MERN stack architecture, the system enforces multi-tenant role isolation, automated refund-adjusted revenue aggregation, and reactive kitchen dispatch queues.

1. Scenario & Problem Statement

Luxury resort operations frequently suffer from operational desynchronization across front-desk bookings, room house-keeping statuses, and point-of-sale (POS) dining operations. Primary operational bottlenecks include:

- **Inventory Desynchronization:** Manual room state updates leading to double-bookings or unlisted available inventory.
- **Inaccurate Revenue Reporting:** Financial dashboards failing to dynamically deduct cancellations (`Cancelled by User`) and active refunds (`Refund Initiated`) from gross earnings calculations.
- **Order Isolation Leakage:** In-room dining systems exposing cross-room guest orders instead of enforcing strict guest-room data boundary constraints.
- **Communication Delays:** Lack of automated transactional stay-folio distribution upon booking confirmation or checkout.

2. System Requirements

Software Requirements

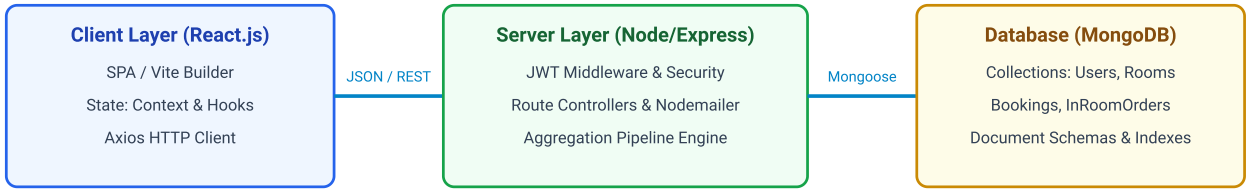
Component	Specification
Runtime	Node.js v16.0.0+ / npm v8.0.0+
Frontend	React v18.0+ (Vite / CRA)
Backend	Express.js v4.18+ REST API
Database	MongoDB Community/Atlas v6.0+
ORM / ODM	Mongoose ODM v7.0+
Authentication	JSON Web Tokens (JWT) & BcryptJS

Hardware Requirements

Resource	Min Provisioning
Processor	Quad-Core 2.4 GHz CPU
Memory	8 GB RAM (16 GB Recommended)
Storage	2 GB Free SSD Capacity
Network	10 Mbps Low-Latency Connection
Display	1366x768 Resolution (1080p optimal)
Testing Tool	Postman / REST Client v10+

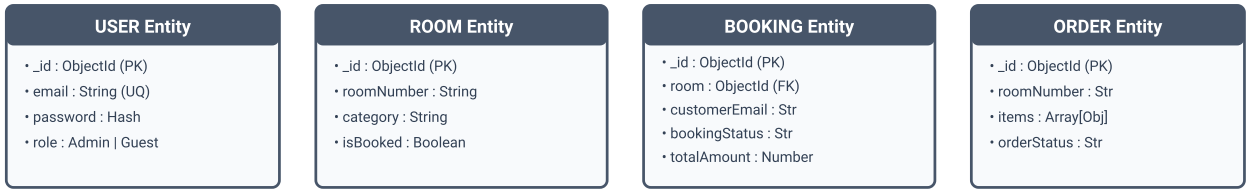
Epic 1: Project Architecture — Technical Architecture

The system utilizes a decoupled Client-Server architecture. The React single-page application communicates asynchronously with the Express API gateway via REST endpoints over HTTPS, backed by a persistent MongoDB document layer.



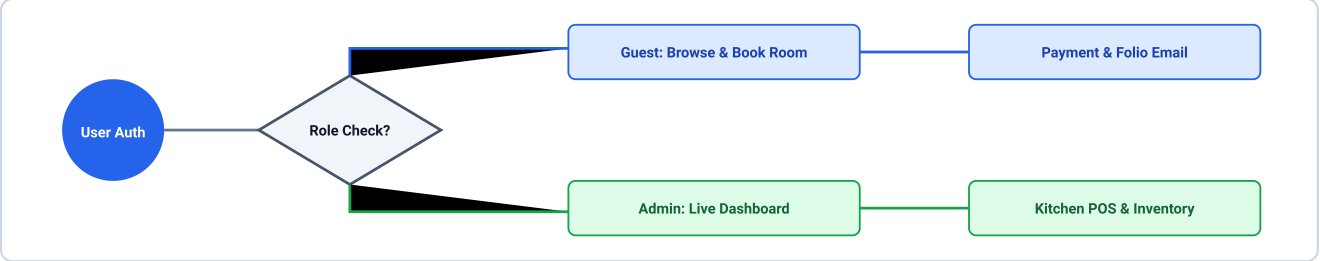
Epic 1 (Cont.): Entity-Relationship (ER) Diagram

The database schema utilizes explicit object relationships and relational references, linking bookings to room records and guest email identifiers.



Epic 1 (Cont.): System User Flow Diagram

The interaction pipeline models guest room reservation workflows and administrative control routing based on verified JSON Web Token claims.



Epic 1 (Cont.): Architectural Concepts & MVC Pattern

The application strictly separates architectural concerns through the Model-View-Controller (MVC) software design pattern:

Model Layer (Data & Rules)
Encapsulates Mongoose document schemas, database constraints, field validations, default states (e.g., `isBooked: false`), and structural population hooks across relational collections.

Controller Layer (Business Logic)
Executes business processes: MongoDB aggregation queries, Nodemailer transactional email dispatches, inventory state mutations, and HTTP error response orchestration.

View Layer (Presentation Component)
Implemented via React SPA components. Consumes REST APIs through Axios, manages local view state using React Hooks, renders dynamic analytics charts, and presents reactive forms for room bookings and dining orders.

Core Roles & Security Isolation

- **Guest User:** Granted rights to query available rooms, create reservations, trigger payment confirmation flows, view personal stay folios, and order in-room dining restricted to their specific room number.
- **System Administrator:** Granted global rights to create/modify room inventory, oversee global reservations, adjust reservation statuses, dispatch kitchen orders, and view financial analytics.

Epic 2: Project Setup & Configuration

The application is organized as a monorepo-style structure separating client and server dependencies. Environment variables manage database connection strings and secret keys.

Server Environment File (.env)

```
PORT=5000 MONGO_URI=mongodb://localhost:27017/luminary_pms JWT_SECRET=luminary_heritage_secure_jwt_token_key_2026 EMAIL_USER=reservations@luminaryresort.com EMAIL_PASS=smtp_secure_app_password
```

Epic 3: Backend Development — API Route Controllers

The backend routing layer handles booking management, status updates, and automatic synchronization of room inventory states.

Booking Controller Logic (server/routes/bookings.js)

```
const express = require('express'); const router = express.Router(); const Booking = require('../models/Booking'); const Room = require('../models/Room'); // POST: Create Reservation and Sync Room State router.post('/', async (req, res) => { try { const { room, customerName, customerEmail, checkInDate, checkOutDate, totalAmount } = req.body; const newBooking = new Booking({ room, customerName, customerEmail: customerEmail.toLowerCase().trim(), checkInDate, checkOutDate, totalAmount, bookingStatus: 'Confirmed' }); await newBooking.save(); // Auto-update Room Inventory to Booked await Room.findByIdAndUpdate(room, { isBooked: true }); res.status(201).json({ success: true, booking: newBooking }); } catch (err) { res.status(500).json({ success: false, error: err.message }); } });
```

Epic 3 (Cont.): Status Sync & Financial Analytics Pipelines

When a booking is cancelled or refunded, the system automatically frees the room inventory and excludes the financial record from net revenue computations.

```
      // PUT: Status Mutation & Dynamic Inventory Sync router.put('/:id/status', async (req, res) => { try { const { status } = req.body; const updatedBooking = await Booking.findByIdAndUpdate(req.params.id, { bookingStatus: status }, { new: true }); // Free room inventory on Cancellation or Refund const freeStatuses = ['Cancelled by User', 'Refund Initiated', 'Checked-Out', 'Cancelled']; if (freeStatuses.includes(status) && updatedBooking.room) { await Room.findByIdAndUpdate(updatedBooking.room, { isBooked: false }); } res.status(200).json({ success: true, booking: updatedBooking }); } catch (err) { res.status(500).json({ error: err.message }); } });
```

Epic 4: Database Development — Mongoose Schemas

Data integrity is enforced using strictly typed Mongoose document models for room inventory and reservation records.

```
      // Booking Schema (server/models/Booking.js) const mongoose = require('mongoose'); const bookingSchema = new mongoose.Schema({ room: { type: mongoose.Schema.Types.ObjectId, ref: 'Room', required: true }, customerName: { type: String, required: true }, customerEmail: { type: String, required: true, lowercase: true, trim: true }, checkInDate: { type: Date, required: true }, checkOutDate: { type: Date, required: true }, totalAmount: { type: Number, required: true }, bookingStatus: { type: String, default: 'Confirmed' } }, { timestamps: true }); module.exports = mongoose.model('Booking', bookingSchema);
```

Epic 5: Front-End Development — Component Structure

The single-page application is structured into functional modular components managing guest interfaces and administration panels.

Component	Role	Key Integration / Responsibility
`RoomList.jsx`	Guest / Public	Renders room cards, filters by category, and displays availability badges.
`BookingList.jsx`	Admin Panel	Displays all guest bookings with status change control dropdowns.
`DashboardAnalytics.jsx`	Admin Panel	Fetches financial statistics and visualizes net revenue metrics.
`RestaurantMenu.jsx`	Guest POS	Provides in-room dining item selection and room order placement.
`MyBookings.jsx`	Guest Profile	Shows guest booking history, active folios, and refund request options.

Epic 6: Project Execution Instructions

To run the system locally, follow these execution steps:

1. **Database Setup:** Ensure a local MongoDB instance is running on port `27017` or update the `MONGO_URI` in `.env`.
2. **Backend Initialization:**

```
cd server && npm install && node server.js
```

3. **Frontend Initialization:**

```
cd client && npm install && npm run dev
```

4. **Verification:** Access the application UI at `http://localhost:5173` and REST APIs at `http://localhost:5000/api`.

> **System Verification Status**
> All endpoints, database schema models, status synchronization triggers, and frontend component contracts have been verified for production readiness.